

Error Handling

1. Syntax Error:

```
print("Hello, world!"  
# SyntaxError: unexpected EOF while parsing
```

2. Name Error:

```
print(name)  
# NameError: name 'name' is not defined
```

You're trying to print a variable that hasn't been defined yet. Python raises a `NameError` when it can't find a variable by that name.

3. Type Error:

```
5 + "5"  
# TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

4. Index Error:

```
my_list = [1, 2, 3]  
print(my_list[5])  
# IndexError: list index out of range
```

You're trying to access an index that doesn't exist in the list. Python raises an `IndexError` when you go out of bounds.

5. Attribute Error:

```
num = 42
num.append(5)
# AttributeError: 'int' object has no attribute 'append'
```

The `int` object doesn't have an `append()` method. Python raises an `AttributeError` when you try to use a method or property that doesn't exist for that data type.

Debugging Techniques

1. Using the print functions and f-strings

First, using the `print()` function and f-strings at various points in your code can help you understand the flow and state of variables. For example:

```
def add(a, b):
    result = a + b
    print(f'Adding{a} and{b} gives{result}')
    return result
```

By printing the values of `a`, `b`, and `result`, you can verify that the function behaves as expected.

2. Interactive Debugging with the pdb Module

```
import pdb

def divide(a, b):
    pdb.set_trace()
    return a / b

print(divide(10, 2))
```

3. IDE Debugging Tools

4. Using VS Code Debugger

If you use VS Code, you can set breakpoints in your code and run the debugger to pause execution at those points. Here's how to debug the same `divide` function:

Step 1: Set up your code Create a file called `main.py` with the following content:

```
def divide(a, b):  
    result = a / b  
    return result  
  
print(divide(10, 2))  
print(divide(15, 3))
```

Step 2: Set a breakpoint

1. Click in the gutter (left margin) next to line 2 (`result = a / b`) to set a breakpoint
2. A red dot will appear, indicating the breakpoint is set

Step 3: Start debugging

1. Press `F5` or go to Run > Start Debugging
2. Select "Python File" when prompted
3. The debugger will pause execution at your breakpoint

Step 4: Inspect variables

- Hover over variables to see their current values
- Use the Variables panel on the left to see all local variables
- Use the Debug Console at the bottom to evaluate expressions

Step 5: Step through code

Use the debug toolbar to:

- **Continue (F5):** Resume execution until the next breakpoint
- **Step Over (F10):** Execute the current line and move to the next
- **Step Into (F11):** Enter into function calls
- **Step Out (Shift+F11):** Exit the current function

IDE debugging tools provide a visual interface to examine the state of your program, making it easier to identify and fix issues compared to using print statements alone.

By mastering these foundational debugging techniques - using `print()` statements, the `pdb` module, and IDE tools - you can effectively identify and resolve issues in your Python code. Each technique has its place: `print()` statements for quick checks, `pdb` for interactive exploration, and IDE debuggers for visual inspection.